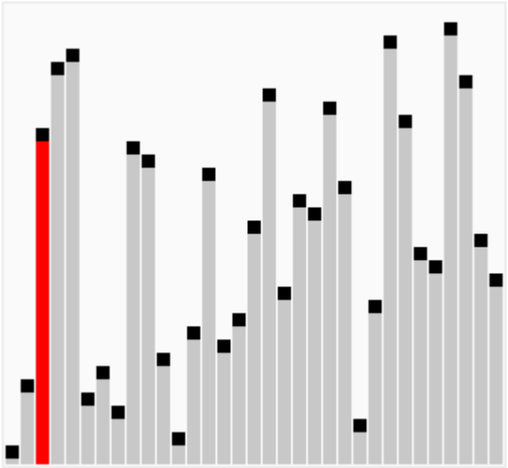
Sorting I

- Selection Sort
- Insertion Sort
- Quicksort
- Mergesort



Computational Problem Sorting

Input: Sequence of numbers $A = \langle a_1, a_2, ..., a_n \rangle$

~~Output: Set of all sorted array (implicitly given)~~

Output ~~(I/O relation)~~**:** Permutation $\langle a'_1, a'_2, ..., a'_n \rangle$ of A with $a'_1 \leq a'_2 \leq ... \leq a'_n$.

Sorting Playing Cards

How do you sort playing cards?

Algorithm?

Selection Sort

Idea:

- Find smallest element and put it to the beginning,
- Find smallest element under the remaining elements,
- Etc.

More precisely:

- Find smallest element, exchange it with the element at the first position,
- Find smallest element in the rest, exchange it with the element at the second position
- Etc

Theorem: SelectionSort sorts any n-element array with runtime $\Theta(n^2)$.

Insertion Sort

Idea:

The beginning (prefix) of the array is sorted.

- Initially, only the first element is sorted, gradually more elements are added.
- Gradually insert each element in its correct position.

```
InsertionSort(array):
for i = 1 to n-1 do
    // Prefix A[0..i] is already sorted
    pos = i
    while (pos > 0) and (A[pos] < A[pos-1]) do
        swap(A[pos], A[pos-1])
        pos = pos - 1
```

Insertion Sort

```
InsertionSort(array):
for i = 1 to n-1 do
    // Prefix A[0..i] is already sorted
    pos = i
    while (pos > 0) and (A[pos] < A[pos-1]) do
        swap(A[pos], A[pos-1])
        pos = pos - 1
```

Runtime T(n)=O(#iterations of the while loop)=:x(n)

- **Worst-case:** $\mathrm{x(n)} \leq O\left(\sum_{i=1}^{n-1} i\right) = O\left(n \cdot \frac{n+1}{2}\right) = O(n^2)$
- **Best-case:** $\mathrm{x(n)} \geq \sum_{i=1}^{n-1} 1 = \mathrm{n} = \Omega(n)$ (presorted →leave while loop immediately)

Theorem: Insertion Sort sorts any n-element array with runtime $\mathrm{O}(n^2)$.
The best-case runtime is $\Omega(n)$.

How good is quadratic runtime?

Quadratic runtime: 2x larger input 4x larger runtime

this is often suboptimal in practice

- Pick an algorithm with $\mathrm{T(n)} \approx \mathrm{n}^2$ basic operations like insertion sort
- Assume 1Ghz CPU with 1 basic operation per CPU cycle

Input size n	Input size Memory	Runtime
10^3	$\approx 4KB$	$(10^{\mathbf{3}})$^$\mathbf{2} \cdot \mathbf{10}$^$-\mathbf{9}$ s=1ms
10^6	$\approx 4\ MB$	$(10^{\mathbf{6}})$^$\mathbf{2} \cdot \mathbf{10}$^$-\mathbf{9}$ s=16.7 minutes
10^9	$\approx 4\ GB$	$(10^{\mathbf{9}})$^$\mathbf{2} \cdot \mathbf{10}$^$-\mathbf{9}$ s=31.7 years

Too slow for large instances!

Is linear runtime much better?

- Pick an algorithm with $\mathbf{T(n)} \approx \mathbf{n}$ basic operations like insertion sort
- Assume 1Ghz CPU with 1 basic operation per CPU cycle

Input size n	Input size Memory	Runtime
10^3	$\approx 4KB$	$10^{\mathbf{3}} \cdot \mathbf{10}\text{^} - \mathbf{9}$ s=~~1ms~~ 10^-6s
10^6	$\approx 4\ MB$	$10^{\mathbf{6}} \cdot \mathbf{10}\text{^} - \mathbf{9}$ s=~~16.7 minutes~~ 10^-3s
10^9	$\approx 4\ GB$	$10^{\mathbf{9}} \cdot \mathbf{10}\text{^} - \mathbf{9}$ s=~~31.7 years~~ 1s

Much much better!

Quick Sort

A faster sorting algorithm

Quicksort: yet another sorting algorithm

Idea (Divide-and-conquer):

- Divide the array into two parts:
- Left part: smaller elements, right part: larger elements.
- Recursively sort both parts!

12	4	8	9	1	16	3	10	5	7	6	14	2	11	13	15

4	8	1	3	5	7	6	2

9	12	16	10	14	11	13	15

sort recursively

sort recursively

1	2	3	4	5	6	7	8

9	10	11	12	13	14	15	16

Quicksort: yet another sorting algorithm

Idea (Divide-and-conquer):

- Divide the array into two parts:
- Left part: smaller elements, right part: larger elements.
- Recursively sort both parts!

We need to decide how to partition the elements into two groups
→ pivot element

Quicksort: Illustration

(choice of pivot element differs from pseudocode)

12	4	8	9	1	16	3	10	5	7	6	14	2	11	13	15

pivot

4	8	1	3	5	7	6	2

9	12	16	10	14	11	13	15

4	1	3	2

8	5	7	6

9	12	10	11

16	14	13	15

1	2	4	3	5	6	8	7	9	10	11	12	14	13	16	15

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

(this example has perfect choices of the pivot elements)

Quicksort: yet another sorting algorithm

Idea (Divide-and-conquer):

- Divide the array into two parts:
- Left part: smaller elements, right part: larger elements.
- Recursively sort both parts!“

```
QUICKSORT(A, low, high):
    if low < high:
        (A,pivot_index) = PARTITION(A, low, high)
        QUICKSORT(A, low, pivot_index - 1)
        QUICKSORT(A, pivot_index, high)
Return A
```

Why do we return the input array A at the end? Why is it sorted?

Partition:

12	4	8	15	1	16	3	10	5	7	6	14	2	11	13	9

i j

Idea: Two variables i and j to iterate through the array from the left and the right. If an element is on the correct side, do nothing and continue (move pointer). If an element needs to switch sides, it can be swapped with an element on the other side.

More detailed:

Increment i until $A[i] > pivot$ (element must move to the right).

Decrement j until A[j]≤pivot (element must move to the left).

Swap the elements, then move i forward and j backward

End: The partition is complete when i and j meet.

Partition: Pseudocode

```
PARTITION(A, low, high):
    pivot = A[high]  // Choose the last element as pivot
    i = low – 1;  j = high + 1
   while i<j
       do i++ while A[i]<pivot
       do j–- while A[j]>pivot
        if i < j
           swap A[i] and A[j]
   return j //Index where the pivot is positioned at the end
```

Choice of Pivot Element

- The size of the two parts during the divide depends on the choice of the pivot.
- The algorithm is fastest when the two parts are as equal as possible.

Strategies for Determining the Pivot:

- ***Ideal: The Median***
  The ideal pivot is the median, but it cannot be easily found.
- ***Fixed Element*** *(e.g., the last element in the range, as we did it)*
  This can lead to very unbalanced parts.
- **Random Element (within the range)**
  Randomized QuickSort typically uses this approach.
  It usually results in reasonably balanced parts. We may look at this later.

Randomization can be extremely helpful. Only glimpse at the topic in this course

Design principle: Divide-and-Conquer

Divide and Conquer:

Common principle for algorithm design

- Divide the input into 2 or more smaller subproblems
- Solve the subproblems recursively
- Combine the partial solutions into the overall solution

Runtime very often satisfies a **recurrence relation**:

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$$

$$T(O(1)) = O(1) \text{ (base cases)}$$

$T(x) =$ runtime for instance of size at most x

Partition: Pseudocode

```
PARTITION(A, low, high):
    pivot = A[high]  // Choose the last element as pivot
    i = low – 1;  j = high + 1
   while i<j
       do i++ while A[i]<pivot
       do j-- while A[j]>pivot
        if i < j
            swap A[i] and A[j]
   return j //Index where the pivot is positioned at the end
```

O(1)

O(|high-low|)
i, j, together run from low to high once

O(1)
Single swap

O(1)

Runtime of partition is in $O(n)$

Runtime Quicksort

Worst Case (when the array is already sorted):

$$
\begin{aligned}
T(n) &= T(n-1) + T(1) + c \cdot n \\
&= T(n-2) + 2 \cdot T(1) + c \cdot (n + n - 1) \\
&= T(n-3) + 3 \cdot T(1) + c(n + n - 1 + n - 2) \\
&= c \sum_{i=1}^{n} i + n \cdot T(1) = O(n^2)
\end{aligned}
$$

- Sorting left subarray
- Sorting right subarray
- Executing partition

Best Case (balanced partition)

$$
\begin{aligned}
T(n) = T\left(\frac{n}{2}\right) + T\left(\frac{n}{2}\right) + c \cdot n \quad &= 2T\left(\frac{n}{2}\right) + c \cdot n \;=\; 4T\left(\frac{n}{4}\right) + c\left(n + 2 \cdot \frac{n}{2}\right) \\
&= 8T\left(\frac{n}{8}\right) + c \cdot \left(n + 2 \cdot \frac{n}{2} + 4 \cdot \frac{n}{4}\right) \\
&= 2^k T\left(\frac{n}{2^k}\right) + c \cdot \sum_{i=0}^{k-1} n \\
&= 2^k T\left(\frac{n}{2^k}\right) + c \cdot k \cdot n = O\big(n \cdot T(1)\big) + c \cdot \log n \cdot n \\
&= O(n \cdot \log n)
\end{aligned}
$$

Interlude: Space Complexity

Memory/Space-complexity

Another central resource for an algorithm is the used memory.

Simplifications:

- We do not distinguish between different types of memory (L1 cache, L2 cache, ram, harddrive, …) when analyzing the memory but only the amount of used memory
- Similarly to the runtime analysis we only map the memory $S(n)$ as a function of the input size (usually worst case)
- Assumption (!): Elementary data (number, symbols, …) requires constant memory

We distinguish between the **auxiliary memory** used besides the **input memory**.

total memory = input memory + auxiliary memory

Space complexity of basic algorithms

Linear Search:
Binary Search (iterative):
Selection Sort:
Insertion Sort:

Space complexity of basic algorithms

Linear Search: O(1) auxiliary memory
Binary Search (iterative): O(1) auxiliary memory
Selection Sort: O(1) auxiliary memory
Insertion Sort: O(1) auxiliary memory

Total memory: $\text{input} + O(1) = O(n)$

Binary Search: Recursive

```
BinarySearchRecursive(A, x, left, right):
    if left > right then
        return -1  // Base case: x not found
     mid ← (left + right) // 2  // Compute middle index
     if A[mid] == x then
        return mid
      else if A[mid] < x then:
        return BinarySearchRecursive(A, x, mid + 1, right)
      else
        return BinarySearchRecursive(A, x, left, mid - 1)
```

Space complexity: Binary Search Recursive

Auxiliary memory: We need to store a constant number of pointers for each call of the recursive function (if we provide the input array A by reference/pointer)

Auxiliary memory of Recursive binary search (worst case): $O(\log n)$

Space complexity of Quicksort

Auxiliary memory $\mathrm{S(n)} = \Theta(\mathrm{recursionDepth})$, because

- We need to remember pointers
- The partition function works **in-place** and re-orders elements in the memory of the original input array
- The array is provided as a reference to function calls

An **in-place algorithm** is an algorithm that performs its computation using only a small, constant amount of auxilliary memory usually $O(1)$ or at most $O(\log n)$ additional space, beyond the input itself. Instead of creating a separate data structure, it modifies the input directly.

Quicksort is usually considered an in-place algorithm even though it does not strictly ($O(1)$ memory) satisfy this definition.

Merge Sort

An optimal sorting algorithm

MergeSort: Idea

We have seen that splitting into two roughly equal parts in Quicksort leads to an almost-linear runtime ($O(n \cdot \log n)$)

How about always splitting in half and sort recursively?

12	4	8	9	1	16	3	10	5	7	6	14	2	11	13	15

Mergesort: Example

12	4	8	9	1	16	3	10	5	7	6	14	2	11	13	15

split in the middle

12	4	8	9	1	16	3	10

sort recursively

1	3	4	8	9	12	10	16

i i

5	7	6	14	2	11	13	15

sort recursively

2	5	6	7	11	13	14	15

j

How to combine the solutions?

1

k k

Mergesort: Example

12	4	8	9	1	16	3	10	5	7	6	14	2	11	13	15

split in the middle

12	4	8	9	1	16	3	10

sort recursively

1	3	4	8	9	12	10	16

i

5	7	6	14	2	11	13	15

sort recursively

2	5	6	7	11	13	14	15

j j

How to combine the solutions?

1	2

k k

Mergesort: Example

12	4	8	9	1	16	3	10	5	7	6	14	2	11	13	15

split in the middle

12	4	8	9	1	16	3	10

sort recursively

1	3	4	8	9	12	10	16

i

5	7	6	14	2	11	13	15

sort recursively

2	5	6	7	11	13	14	15

j

How to combine the solutions?

1	2	3

k

Mergesort: Example

12	4	8	9	1	16	3	10	5	7	6	14	2	11	13	15

split in the middle

12	4	8	9	1	16	3	10

sort recursively

1	3	4	8	9	12	10	16

5	7	6	14	2	11	13	15

sort recursively

2	5	6	7	11	13	14	15

How to combine the solutions?

1	2	3	4

Mergesort: Example

12	4	8	9	1	16	3	10	5	7	6	14	2	11	13	15

split in the middle

12	4	8	9	1	16	3	10

sort recursively

1	3	4	8	9	12	10	16

5	7	6	14	2	11	13	15

sort recursively

2	5	6	7	11	13	14	15

How to combine the solutions?

1	2	3	4	5

Mergesort: Example

12	4	8	9	1	16	3	10	5	7	6	14	2	11	13	15

split in the middle

12	4	8	9	1	16	3	10

sort recursively

1	3	4	8	9	12	10	16

5	7	6	14	2	11	13	15

sort recursively

2	5	6	7	11	13	14	15

How to combine the solutions?

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

MergeSort: Algorithm

Merge Sort: Core Idea Divide-and-Conquer Strategy

- **Divide:** Split the array into two halves (arbitrary split!).
- **Conquer:** Recursively sort each half using the same procedure.
- **Combine:** Merge the two sorted halves into a single sorted array.

Key Insight

Two **sorted** sequences can be merged **efficiently** by repeatedly selecting the smallest front element.

Mergesort: Pseudocode 1

```
MERGESORT(A, left, right):
    if left < right:
        mid = (left + right) / 2
        MERGESORT(A, left, mid)
        MERGESORT(A, mid + 1, right)
        MERGE(A, left, mid, right)
```

Merge: Pseudocode 1

```
MERGE(A, left, mid, right):
    L = A[left : mid + 1]  // Left subarray
    R = A[mid + 1 : right + 1]  // Right subarray
     i = 0, j = 0,    k = left  // Index for merged array
     while i < length(L) and j < length(R):
        if L[i] ≤ R[j]:
            A[k] = L[i]
            i = i + 1
        else:
            A[k] = R[j]
            j = j + 1
        k = k + 1
```

Merge: Pseudocode 2

```
MERGE(A, left, mid, right):
    …
     // Copy any remaining elements
    while i < length(L):
        A[k] = L[i]
        i = i + 1
        k = k + 1

    while j < length(R):
        A[k] = R[j]
        j = j + 1
        k = k + 1
```

MergeSort: Correctness (Sketch)

Induction on input size n

- **Base Case**: $n \leq 1$
  Array already **sorted and elements preserved**
- **Inductive Step**
  Split array → **two subarrays**
  Recursive calls correctly **sort** both halves *(induction hypothesis)*

  Key Lemma for Induction Step: Correctness of Merge
  Inputs: two sorted arrays
  Output: (1) sorted and (2) contains all elements of both inputs
  Argument: Two pointers, Always take **smallest front element**
  (formally with yet another loop invariant)

Mergesort: Runtime Analysis

$T(1) = O(1),\ T(n) = 2 \cdot T\left(\frac{n}{2}\right) + c \cdot n,$

- We always split the array in half
- $c \cdot n$ is the runtime of the Merge operation.
  Why is it linear in the size of the elements that need to be merged?

The number of basic operations is proportional to the number of pointer movements. The i pointer moves at most $n/2$ times. The j pointer moves at most $n/2$ times. The k pointer moves at most n times. Thus there are at most $2n$ pointer movements.

We have seen a recurrence relation as above when analyzing the best case complexity of quicksort $T(n) = O(n \cdot \log n)$

Mergesort: Memory Analysis

- Keeping track of the pointers for recursion requires auxiliary space
$$O(recursionDepth) = O(\log n)$$
- The merge operation requires $O(n)$ auxiliary space (the k pointer moves in a separate array). Mergesort is not in-place!
  But, we can re-use this space on different levels of the recursion

Total Space: $\text{input} + \ O(n + \log n) = O(n)$